

YOLO Final 检查点 8 报告

模型导出、推理服务与速度测试

2026-05-13

任务目标

本检查点聚焦模型导出、推理服务与基础量化处理。本轮工作基于当前已经完成训练和正式评估的最佳策略。

- 配置: configs/dual_scale_three_box_coco_only_noobj1_416_basic_aug_scale_jitter_50_lr7e4.toml
- checkpoint: outputs/dual_scale_three_box_coco_only_noobj1_416_basic_aug_scale_jitter_50_lr7e4_ddp_20260512_130823/best.pth
- 模型: deep_residual_dual_scale_three_box, 416 输入, P4/P5 双尺度, 三框检测头。
- 训练口径: COCO-only, basic augmentation + scale jitter, 50 epoch, 学习率 0.0007。
- 正式 COCO 指标: AP 0.192170, AP50 0.310452, AP75 0.200965, APS 0.084709, AR100 0.351947。

现有前后端接入

项目中已经存在可用的前后端。后端为 Flask REST API, 前端通过 /model_predict 调用推理服务。因此, 本轮没有重写服务框架, 而是在后端 predictor 预留位置补充 PyTorch、TorchScript 与 ONNX 三种格式切换。

模块	文件	说明
后端	backend/app.py	Flask REST API
前端	frontend/app.js	调用 /health 和 /model_predict
共享运行时	backend/exported_predictor_utils.py	预处理、解码、NMS、响应格式
TorchScript	backend/torchscript_predictor.py	加载导出 .pt
ONNX	backend/onnx_predictor.py	加载 ONNX Runtime session

导出实现

新增 tools/export_model.py。模型原始输出为多尺度字典, 导出时通过轻量 wrapper 转换为稳定 tuple 输出, 输出顺序为 p4、p5。后处理保留在 Python 侧, 保证 PyTorch、TorchScript 和 ONNX 对比时只替换模型前向部分。

```
python tools/export_model.py \
--formats torchscript,onnx \
--output-dir exports/checkpoint8 \
--prefix best_yolofinal_416_lr7e4 \
--device cuda \
--batch-size 1
```

产物	路径/状态
TorchScript	exports/checkpoint8/best_yolofinal_416_lr7e4.torchscript.pt
大小	139.3 MB
metadata	exports/checkpoint8/best_yolofinal_416_lr7e4.export_metadata.json
ONNX	onnx package is not installed; skipping ONNX export.

TorchScript 导出后与 PyTorch wrapper 的 raw tensor 输出做了数值检查: P4 max/mean abs diff = 0.074009/0.005468; P5 max/mean abs diff = 0.036488/0.006601。

速度测试

新增 tools/benchmark_exported_model.py。测试使用 64 张 COCO val 图像, warmup 8 张, score threshold 0.5, top-k 10, NMS IoU threshold 0.5, 且所有格式共用同一套 Python 后处理。

格式	样本	推理均值 ms	总耗时均值 ms	P95 总耗时	端到端 FPS
PyTorch .pth	64	4.003	6.643	7.367	150.54
TorchScript	64	2.954	5.587	6.295	178.98

TorchScript 相比原始 PyTorch checkpoint，模型前向平均耗时降低约 26.2%，端到端平均耗时降低约 15.9%，端到端 FPS 从 150.54 提升到 178.98。

ONNX 后端代码已经接入，但当前 yolov1 环境缺少 onnxruntime；ONNX 导出也因为缺少 onnx 包被跳过。这属于环境依赖未安装，不是模型结构不支持。

INT8 PTQ 量化

本轮补充了 CPU 侧 FX graph mode PTQ 静态量化。直接全模型量化会让 decoupled head 中 reg/cls cat 的量化尺度不一致，因此正式产物采用更稳妥的策略：backbone INT8，detection head 保持 FP32。校准数据使用 128 张 COCO val 图片。

```
python tools/quantize_model.py \
--calibration-samples 128 \
--output-dir exports/checkpoint8 \
--prefix best_yolofinal_416_lr7e4_int8_backbone_calib128 \
--backend x86
```

产物	数值
INT8 TorchScript	exports/checkpoint8/best_yolofinal_416_lr7e4_int8_backbone_calib128.torchscript.pt
文件大小	54.3 MB
校准样本	128 张 COCO val
量化后端	x86
保持 FP32	.*head.*

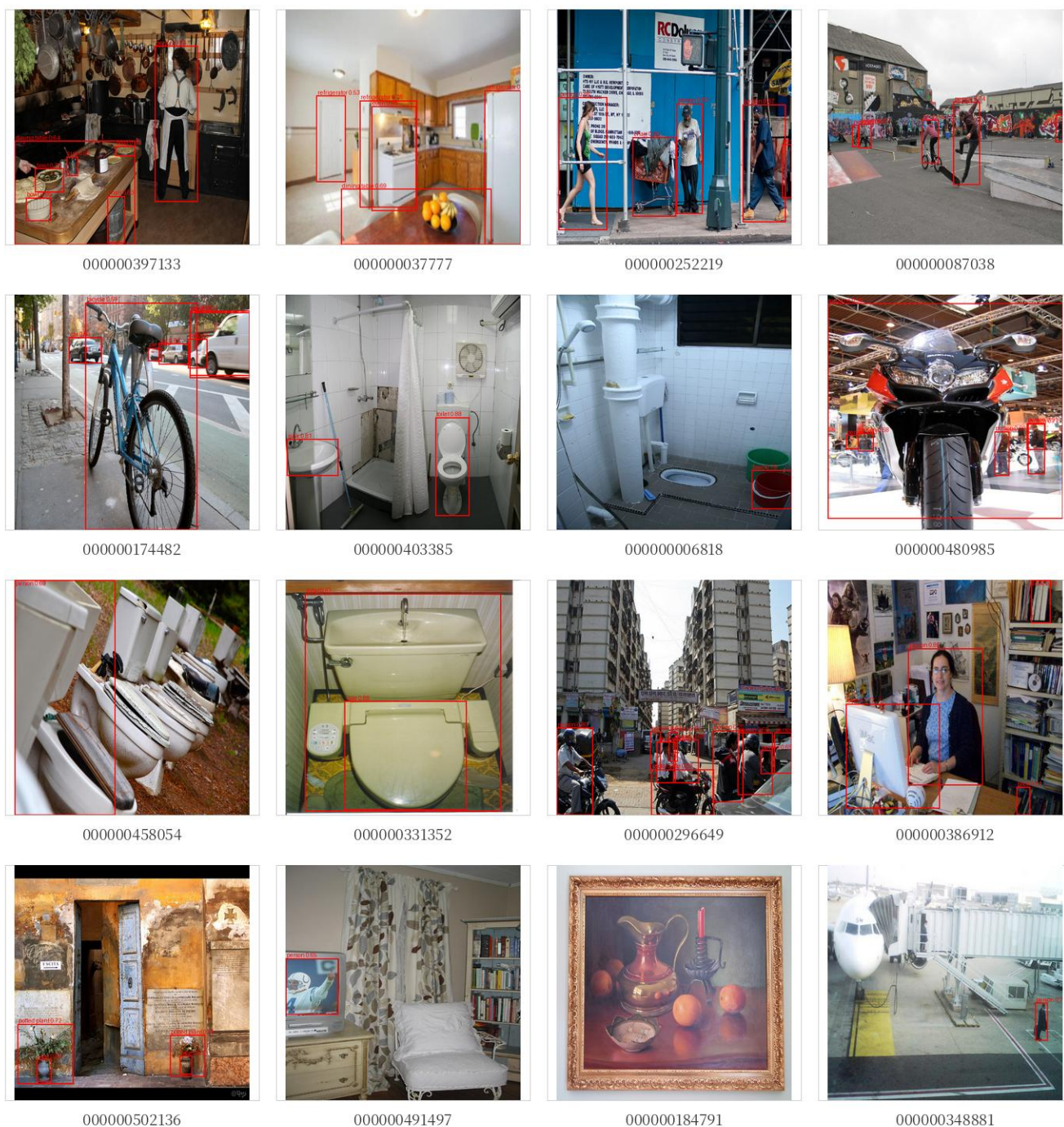
CPU 格式	样本	推理均值 ms	总耗时均值 ms	P95 总耗时	端到端 FPS
PyTorch .pth	64	61.616	64.702	72.828	15.46
INT8 TS	64	8.993	11.823	12.957	84.58

在 CPU 端，INT8 TorchScript 的端到端 FPS 为 84.58，PyTorch CPU baseline 为 15.46；端到端平均耗时降低约 81.7%。该结果说明量化产物适合 CPU 推理服务演示。

模型	COCO AP	AP50	AP75	AR100
FP32 best	0.192170	0.310452	0.200965	0.351947
INT8 backbone	0.191483	0.309759	0.199871	0.351494

量化模型完整 COCO val 评估使用 5000 张图片和正式 score 设置。相比 FP32 best，AP 下降约 0.000687，说明 backbone INT8 + head FP32 的 PTQ 策略在当前模型上精度损失很小。

16 张推理可视化



以上图片由 TorchScript 后端生成，红框为预测框，标签包含类别名和置信度。展示参数与速度测试一致：score threshold 0.5，top-k 10，NMS IoU threshold 0.5。

复现记录

- 当前 Git commit: 1c0c2c1。
- 本轮新增或修改的核心文件: tools/export_model.py、tools/benchmark_exported_model.py、backend/exported_predictor_utils.py、backend/torchscript_predictor.py、backend/onnx_predictor.py、backend/pytorch_predictor.py、backend/app.py、backend/config.py、backend/README.md。
- 导出产物: exports/checkpoint8/best_yolofinal_416_lr7e4.torchscript.pt。
- 测速记录: outputs/export_benchmark/checkpoint8_lr7e4_benchmark.json。
- 16 张可视化: outputs/export_benchmark/checkpoint8_lr7e4_vis16/torchscript/。
- INT8 量化产物: exports/checkpoint8/best_yolofinal_416_lr7e4_int8_backbone_calib128.torchscript.pt。

- INT8 速度记录: `outputs/export_benchmark/checkpoint8_lr7e4_int8_backbone_calib128_cpu_benchmark.json`。
- INT8 COCO eval: `outputs/evaluations/checkpoint8_lr7e4_int8_backbone_calib128_coco_eval.json`。
- 当前工作区仍包含实验输出和报告文件的未提交状态; 正式归档时应将代码、配置、导出脚本、测速 JSON、展示图和 PDF 一起保存。

结论

检查点 8 的核心目标已经完成: 在已有前后端框架中接入导出模型推理路径, 并完成 TorchScript 导出、速度测试和 16 张图片可视化。TorchScript 后端在不改变前端接口、不改变 Python 后处理逻辑的前提下, 将端到端 FPS 从 150.54 提升到 178.98, 是当前可直接用于 GPU/常规导出演示的方案。补充的 INT8 PTQ 产物在 CPU 上将端到端 FPS 提升到 84.58, 适合 CPU 推理服务展示。